

PRÁCTICA 2: RERCURSIVIDAD

Rodrigo García Carmona
rodrigo.garcia@upm.es



Esta práctica es individual y sirve tanto para desarrollar como para poner a prueba los conocimientos adquiridos. Realizar esta práctica es recomendable para aumentar las probabilidades de éxito en los exámenes teóricos.

1. Preparación de la práctica

El objetivo de esta práctica es interiorizar el concepto de recursividad y aprender cómo implementar métodos recursivos. Se estudiarán los siguientes conceptos:

- Comprensión del concepto de recursividad.
- Análisis de un algoritmo recursivo.
- Implementación de un algoritmo recursivo.
- Análisis elemental del rendimiento de un programa.

TRABAJO PREVIO AL LABORATORIO

El tiempo de las clases de laboratorio es muy valioso, ya que en ellas tendrá a su disposición a los profesores para resolver cualquier duda que le pueda surgir. Por tanto, para facilitar al máximo el aprovechamiento de este tiempo, las prácticas se han dividido en varias secciones. La primera de ellas (“preparación de la práctica”) puede ¡y debe! realizarse **antes de la sesión de laboratorio** propiamente dicha. Leer esta sección antes de entrar a clase hará que su tiempo en el laboratorio sea más productivo y provocará que las dudas más importantes le surjan **durante la clase**, y **no después**, cuando se encuentre estudiando solo en su casa y la única opción que le quede sea escribir un email al profesor.

De hecho, es especialmente importante evitar que se den situaciones como esta la noche antes de un examen.

1.1. Exoesqueletos

La robótica tiene numerosas aplicaciones en el campo de la ingeniería biomédica, pero para esta práctica nos centraremos en una concreta: los exoesqueletos. Estos dispositivos tienen un potencial enorme para mejorar la calidad de vida de sus usuarios, ya que pueden ayudar en un proceso de recuperación tras una lesión neurológica o servir para compensar hasta cierto punto una discapacidad. Estas aplicaciones de los exoesqueletos reciben el nombre de “neurorehabilitación” y “asistencia a la discapacidad”, respectivamente:

- **Neurorehabilitación:** los exoesqueletos usados en procesos de neurorrehabilitación están diseñados para ayudar a los pacientes a recuperar la función motora después de un problema neurológico, como puede ser un accidente cerebrovascular (como un ictus) o una lesión de la médula espinal. Estos dispositivos son capaces de guiar las extremidades del paciente, ayudándole a realizar el conjunto de movimientos repetitivos que conforman su terapia. La principal ventaja que ofrecen respecto a un fisioterapeuta humano es que permiten al paciente realizar la terapia durante más tiempo, con mayor consistencia y con mejor precisión. Además, los exoesqueletos también pueden registrar los datos biomecánicos del paciente durante todo el proceso, permitiendo así a los profesionales médicos monitorizar sus avances.

- **Asistencia a la discapacidad:** los dispositivos robóticos de asistencia a la discapacidad están diseñados para el uso diario, ayudando a las personas con discapacidades motoras a realizar actividades como caminar, levantar objetos, mantener una higiene básica o alimentarse. Estos dispositivos no buscan necesariamente “curar” la condición que sufre el paciente (en muchos casos, esto ni siquiera es posible), sino simplemente compensar la función perdida y dotarle de una independencia que sirva para mejorar su calidad de vida.

Un exoesqueleto robótico es una estructura externa que se coloca sobre el cuerpo de una persona. Un exoesqueleto activo (es decir, que complementa o sustituye la fuerza del paciente), tiene las siguientes partes:

- Una **estructura esquelética** (el exoesqueleto propiamente dicho) que se compone de “huesos” o segmentos conectados mediante “articulaciones”, y que está diseñado para poder realizar un tipo de movimientos concreto. El objetivo de esta estructura es aumentar la fuerza o restaurar la movilidad de las extremidades, y su diseño siempre está inspirado en la propia biomecánica del cuerpo humano, alineando sus articulaciones con las del usuario (cadera, rodilla, codo, etc.).
- Un conjunto de **motores** (también llamados “actuadores”) que hacen posible el movimiento del exoesqueleto sin tener que recurrir únicamente a la fuerza del usuario, ya sea empleando una pequeña parte de esta o prescindiendo por completo de ella.
- Varios **sensores** que determinan en qué posición se encuentra cada una de las partes del exoesqueleto o miden parámetros fisiológicos del usuario.
- Un **sistema de control** que, a partir de la información recabada por los sensores y los objetivos terapéuticos que se desean alcanzar (moverse de determinada forma o asistir el movimiento del usuario) calcula qué órdenes concretas deben darse a los motores para que el exoesqueleto ejecute el movimiento en cuestión.

La Figura 1 ilustra cómo un exoesqueleto real (en este caso, el **HYBRID**) integra todos estos componentes. En la parte izquierda de la figura puede ver la estructura esquelética e, insertada en ella, tres articulaciones para cada pierna. Una de estas articulaciones, la rodilla, aparece desglosada en la parte derecha de la figura, pudiéndose ver que está compuesta de un motor (*EC60 motor*), un sensor (*potentiometer*) y la electrónica necesaria para comunicarla con el sistema de control (*dsPIC 30F4013*).

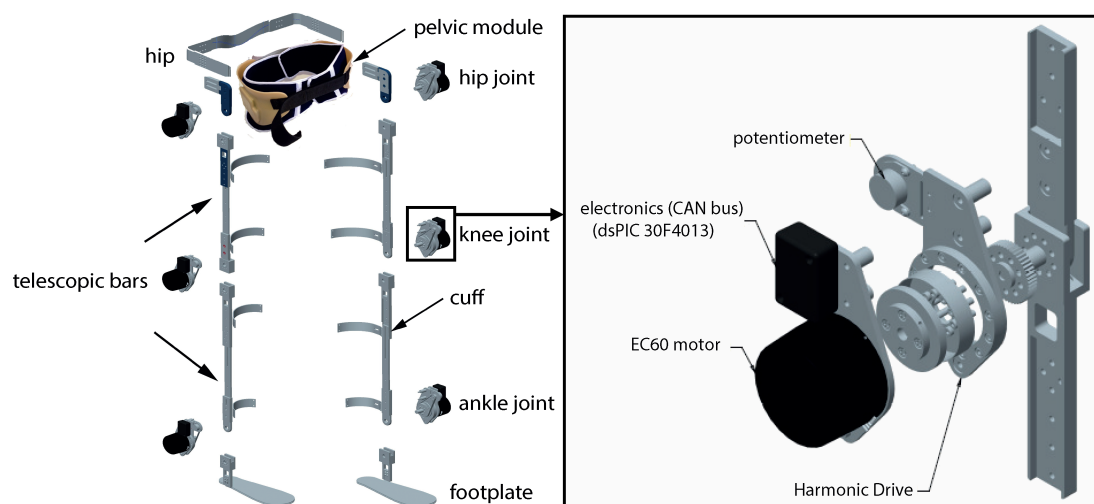


Figura 1: Estructura del exoesqueleto HYBRID¹

La Figura 2 muestra cómo se lleva a cabo, en este mismo dispositivo, la comunicación entre las articulaciones y los sistemas de control, que se ejecutan en un pequeño ordenador (*PC-104*).

¹Eloy Urendes, Guillermo Asín-Prieto, Ramón Ceres, Rodrigo García-Carmona, Rafael Raya y José L. Pons.

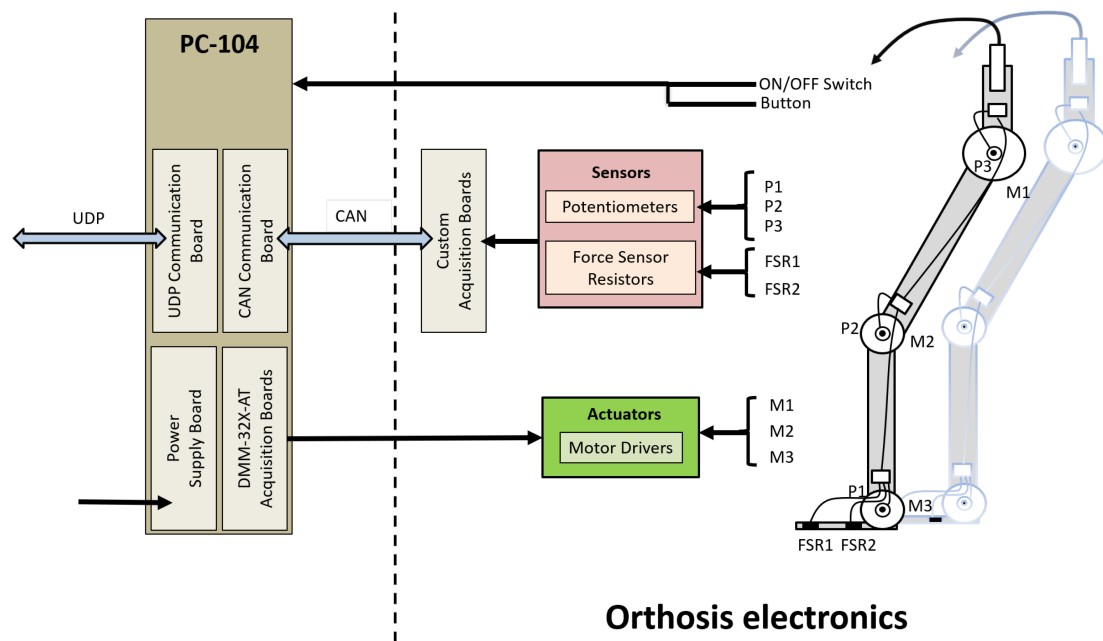


Figura 2: Conexión del exoesqueleto HYBRID con su sistema de control¹

PRÓTESIS ROBÓTICAS

Las prótesis, es decir, la sustitución de un miembro perdido por uno artificial, han existido desde hace siglos, pero gracias a la robótica es posible construir prótesis “biónicas”, que utilizan motores, microprocesadores y componentes electrónicos para imitar de forma mucho más fiel la funcionalidad de la extremidad perdida.

De hecho, no hay mucha diferencia desde un punto de vista ingenieril entre el desarrollo de una prótesis o de un exoesqueleto: ambos son dispositivos robóticos que utilizan sensores, motores y un sistema de control para ejecutar un movimiento concreto. La diferencia fundamental es que los exoesqueletos se sitúan sobre el cuerpo humano y las prótesis sustituyen alguna parte del mismo.

El funcionamiento de un exoesqueleto exige un control sumamente preciso, pues estamos trabajando con motores capaces de ejercer una gran fuerza, suficiente para hacer daño al paciente si le obligan a asumir una postura incorrecta. Y aquí es donde entra en juego el sistema de control del que se hablaba antes. Este sistema es el cerebro del exoesqueleto, el responsable de decidir cómo deben moverse los motores siguiendo un algoritmo programado en *software*.

1.2. Cinemática directa

Uno de los problemas más comunes que el *software* del sistema de control de un exoesqueleto debe resolver es el de la cinemática directa (*forward kinematics* en inglés). Este proceso consiste en calcular la **posición y orientación** del extremo final del exoesqueleto (llamado en inglés *end-effector*) a partir de las **características de sus segmentos** (los “huesos”) y los **ángulos** que estos forman.

CINEMÁTICA

Recordará, de sus clases de física, que la cinemática es la rama de la mecánica que describe el movimiento de los objetos sin tener en cuenta las fuerzas que lo causan. Por tanto, en esta práctica lo único que tendremos en cuenta será la posición en el espacio de cada una de las partes del exoesqueleto.

Imaginemos el exoesqueleto de un brazo, compuesto por una **cadena** de varios segmentos: un brazo, un antebrazo y una mano, en la que el brazo está conectado con el antebrazo, y el antebrazo a su vez con la mano. Si conocemos:

- los parámetros geométricos de cada uno de los segmentos,
- el ángulo que forman con el segmento anterior de la cadena, y
- la posición y orientación absolutas del primer segmento de la cadena,

la cinemática directa nos proporciona un conjunto de ecuaciones con las que calcular la ubicación precisa del extremo final de la mano en un sistema de coordenadas tridimensional (x, y, z) , así como su orientación (α, β, γ) .

ORIENTACIÓN Y SISTEMAS DE COORDENADAS

Los tres ángulos de la orientación reciben diferentes denominaciones en el mundo de la computación y en el de la medicina. En el de la computación se conocen como “balanceo”, “cabeceo” y “guiñada”, términos heredados de la aeronáutica. En el de la medicina se trabaja con los ángulos de los planos anatómicos, siendo estos el “sagital”, el “frontal” y el “transversal”.

Para complicar aún más este asunto, hay que tener en cuenta que los sistemas de coordenadas tridimensionales pueden seguir la regla de la mano izquierda o la de la mano derecha. Si siente curiosidad, investigue todo esto.

El control de movimientos es la aplicación más directa de la cinemática directa. Por ejemplo: para que un exoesqueleto guíe la pierna de un paciente a través de una trayectoria de marcha específica, el sistema de control debe recurrir constantemente a la cinemática directa para verificar que la posición del pie del exoesqueleto coincide con la posición deseada en cada instante. Si detecta un error, deberá ajustar los motores de las articulaciones para corregirlo.

De hecho, este proceso puede realizarse antes de ejecutar el movimiento, verificando así que este no hará que el exoesqueleto colisione consigo mismo o con el entorno. También permite definir un espacio de trabajo seguro que el exoesqueleto no deberá abandonar bajo ningún concepto, para así proteger al usuario y a los que le rodean. Recuerde que un exoesqueleto puede ejercer mucha fuerza.

En esencia, la cinemática directa es el puente entre los comandos enviados por el sistema de control a los motores (los ángulos de las articulaciones) y el comportamiento resultante del dispositivo en el mundo real. Sin ella, sería imposible lograr un control preciso y seguro en la robótica biomédica.

1.3. Resolución del problema de la cinemática directa en 3 dimensiones

El método más común para resolver el problema de la cinemática directa emplea **matrices de transformación homogénea**, que describen la relación espacial entre un segmento de la cadena y el siguiente.

Una matriz de transformación homogénea es una matriz 4×4 que puede representar tanto la rotación como la traslación de un marco de referencia con respecto a otro en una sola operación. Para un exoesqueleto con n segmentos, la posición y orientación del extremo final con respecto al primer segmento se calcula multiplicando las matrices de transformación de cada segmento en secuencia:

$$T_{final} = T_1 \cdot T_2 \cdot T_3 \cdot \dots \cdot T_n$$

Donde cada matriz T_i depende del ángulo del segmento θ_i y de sus características geométricas (como su longitud o su forma). El resultado, T_{final} , es una matriz que contiene las coordenadas (x, y, z) de la posición del extremo final y los datos que definen su orientación. Una de las convenciones más utilizadas para definir estas matrices es la de Denavit-Hartenberg, pero no es la única.

Si, llegados a este punto, está asustado (el álgebra suele producir este efecto en los estudiantes), no se preocupe: para la práctica utilizaremos un modelo de cinemática inversa más sencillo, en 2 dimensiones, por lo que los cálculos serán más sencillos.

1.4. Resolución del problema de la cinemática directa en 2 dimensiones

Consideraremos una versión simplificada del modelo de cinemática directa, en el que solo se trabaja con dos coordenadas (x, y) , y los segmentos son siempre rectos y tienen una longitud fija. De cada segmento conocemos:

- La **longitud** del segmento.
- El **ángulo** que forma con su segmento padre (al que está conectado).
- Sus **segmentos hijos** (los que están conectados a él).

VARIOS HIJOS

Importante: Recuerde que cada segmento puede tener varios hijos. Piense cómo el segmento que representa su muñeca está unido a cinco segmentos que representan los metacarpos de cada dedo.

Además, también conocemos la posición (x, y) del **punto inicial** del **segmento raíz**: el único segmento que no tiene un segmento padre.

Para calcular la posición (x, y) del **punto final** de un segmento concreto (s_i) se utilizarán la siguientes ecuaciones, obtenidas mediante trigonometría:

$$x_i = x_{i-1} + L_i \cos(\theta_1 + \dots + \theta_i)$$

$$y_i = y_{i-1} + L_i \sin(\theta_1 + \dots + \theta_i)$$

En las que x_{i-1} e y_{i-1} son la posición del punto final del segmento padre (o, si se trata del segmento raíz, de su punto inicial, pues no tiene padre), y θ son los ángulos de la cadena de segmentos que conducen desde el segmento raíz hasta el segmento actual (inclusive). Si, por ejemplo, el segmento que deseamos calcular (s_3) está conectado de la siguiente forma: $s_3 \rightarrow s_2 \rightarrow s_1$ y s_1 es el segmento raíz:

- θ_1 : Ángulo del segmento raíz (s_1).
- θ_2 : Ángulo del segmento (s_2) conectado al segmento raíz (s_1).
- θ_3 : Ángulo del segmento (s_3) conectado al segmento (s_2) conectado al segmento raíz (s_1).

Las ecuaciones quedarían:

$$x_3 = x_2 + L_3 \cos(\theta_1 + \theta_2 + \theta_3)$$

$$y_3 = y_2 + L_3 \sin(\theta_1 + \theta_2 + \theta_3)$$

Para comprobar que ha entendido cómo resolver este problema, **realice, de forma manual (sin programarlo), el siguiente ejercicio:** Considere una cadena cinemática compuesta de los siguientes segmentos:

- **Segmento 1 (raíz):** En la posición $(0, 0)$, de longitud 4, formando un ángulo de 45° .
- **Segmento 2:** Hijo del segmento 1, de longitud 2, formando un ángulo de 45° .
- **Segmento 3:** Hijo del segmento 2, de longitud 1, formando un ángulo de 90° .
- **Segmento 4:** Hijo del segmento 2 (¡sí, del 2!), de longitud 1, formando un ángulo de -90° .

¿Cuál es la posición del extremo final de los segmentos 3 y 4?

¡No pase a la página siguiente hasta haber terminado! ¡No haga trampas!

...

Si ha hecho bien los cálculos, deberían haber obtenido el siguiente resultado:

$$(x_3, y_3) = (2\sqrt{2} - 1, 2\sqrt{2} + 2)$$

$$(x_4, y_4) = (2\sqrt{2} + 1, 2\sqrt{2} + 2)$$

RECURSIVIDAD

Alguien de mente astuta como usted sin duda habrá sospechado que el problema de la cinemática directa tiene algo que ver con la recursividad, pues para eso forma parte de esta práctica. Repase los cálculos que ha hecho para resolver el ejercicio (porque lo ha resuelto, ¿verdad?) y fíjese en que para poder calcular la posición de un segmento primero es necesario haber calculado la de su padre. Y, para la calcular la posición del segmento padre, será necesario calcular la del “abuelo”, y así sucesivamente...

Efectivamente, una cadena cinemática es una estructura que se presta de forma natural a la recursividad. La dependencia secuencial entre segmentos es inherentemente recursiva. Además, un algoritmo recursivo podrá calcular la cinemática directa para un número arbitrariamente grande de segmentos. En lo que a la implementación del algoritmo respecta, tanto da que nuestro exoesqueleto tenga 3 como 300 segmentos.

1.5. Objetivo y descarga de la práctica

El objetivo de la práctica es terminar de desarrollar una aplicación Java que permite visualizar la posición de los segmentos de un exoesqueleto en un espacio bidimensional. Además, esta aplicación también muestra el movimiento de dicho exoesqueleto de forma animada, permitiendo cargar la secuencia de movimientos desde un archivo.

La práctica está disponible en GitHub, en la siguiente dirección de Internet (URI):

<https://github.com/rgarciacarmona/ALED-lab2>

Para trabajar en ella, siga estos pasos:

1. Haga un *fork* de este repositorio a su cuenta de GitHub.
2. Antes de empezar a hacer *commits*, compruebe en *Settings* → *General* de su *fork* que la visibilidad es **pública**. Solo se evaluarán los *commits* hechos sobre un *fork* público.
3. Clone su *fork* e impórtelo en Eclipse. Para ello, seleccione *File* → *Import...* y, de entre las opciones disponibles, *Git* → *Projects from Git* → *Clone URI*. A continuación, introduzca la URI de su *fork* (<https://github.com/usuario/ALED-lab2>, donde *usuario* es su nombre de usuario de GitHub), no la del repositorio original. Después, haga click en *Next* tantas veces como sea necesario y, por último, en *Finish*.
4. Trabaje, haga *commit* de sus cambios y haga *push* a su *fork*.

Tómese su tiempo para examinar todas las clases del proyecto que acaba de importar. Intente figurarse, a base de observar el código, para qué sirve cada una de ellas.

1.6. Modelado de datos

A la hora de resolver un problema (ya sea biomédico o de cualquier otro ámbito) programando, el primer paso será modelar los datos con los que vamos a tener que trabajar. Dicho de otra forma: tenemos que pensar cómo podemos representar la información que es importante para nosotros en un formato que el ordenador entienda.

En lo que a esta práctica respecta, los segmentos estarán representados mediante la clase `Segment` del paquete `es.upm.aled.lab2.kinematics`, que está **sin implementar**. Esta clase deberá contener toda la información de la que disponemos **para cada segmento**. A saber:

- La **longitud** del segmento, en cm. La modelaremos mediante un `double` al que llamaremos `length`.
- El **ángulo que forma con su segmento padre**, en radianes. Lo modelaremos usando un `double` al que llamaremos `angle`.

- Sus **segmentos hijos**. Los modelaremos como usando una `List<Segment>` a la que llamaremos `children`.

No será necesario almacenar la posición del punto inicial del segmento raíz. Asumiremos que será el origen de coordenadas o, en caso contrario, que nos la proporcionará quien use la clase.

Además, la aplicación cuenta con una interfaz gráfica que permitirá representar los segmentos en forma de un esqueleto capaz de animarse. Este esqueleto está formado por nodos: objetos de la clase `Node` del paquete `es.upm.aled.lab2.gui`.

Node tiene los siguientes atributos:

- `double x` y `double y`: las coordenadas del nodo. El programa dibujará un punto aquí.
- **`List<Node>children`: los nodos conectados con este**. El programa dibujará una línea desde las coordenadas de este nodo hasta cada uno de los nodos a los que esté conectados..

La Figura 3 muestra ambas clases.

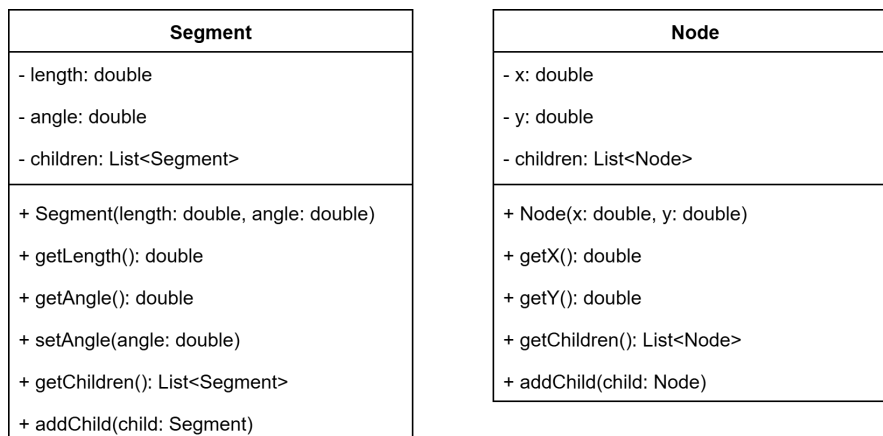


Figura 3: Diagrama de clases del modelo de datos de la práctica

Responda a las siguientes preguntas:

- ¿Tienen `Segment` y `Node` métodos *getters*? ¿Cuáles son?
- ¿Tienen `Segment` y `Node` métodos *setters*? ¿Cuáles son?
- ¿Son `Segment` y `Node` tipos de datos recursivos?

Enhorabuena, ya ha terminado con el trabajo previo a la sesión de laboratorio. Las secciones de la 2 en adelante puede realizarlas en clase.

2. Implementación de la clase Segment

En primer lugar, programará la clase `Segment` del paquete `es.upm.aled.lab2.kinematics`, que ya está creada pero que se encuentra vacía. Esto le permitirá interiorizar la estructura de esta clase, que tendrá que manejar posteriormente (en la sección 4), cuando tenga que implementar un algoritmo recursivo usándola.

2.1. Implementación de los métodos y atributos

Observe la Figura 3, y fíjese en los métodos y los atributos de la clase `Segment`. **Impleméntelos**. En cada caso, el nombre del método deberá ser lo bastante autoexplicativo como para que pueda deducir qué debe hacer cada uno. No obstante, en el caso de uno de los métodos (`addChild()`) hay un aspecto importante que debe tener en cuenta y que no resulta obvio se deduce inmediatamente a partir del nombre: **no debe añadirse un nuevo `Segment` a la lista de hijos si este ya se encontraba en ella**. De no hacerse esta comprobación, podríamos acabar con una cadena cinemática con elementos duplicados.

Si no sabe por dónde empezar, **estudie el funcionamiento de la clase `Node`**, que se encuentra en el paquete `es.upm.aled.lab2.gui`, ya que es similar en muchos aspectos a la que tiene que implementar usted.

2.2. Documentación del código

Ahora que ha terminado de programar la clase `Segment`, es hora de documentarla. Añada todos los comentarios que considere necesarios. En caso de duda, siempre es mejor pasarse que quedarse corto.

Y sí, hemos dicho antes que el nombre de los métodos debería ser autoexplicativo a la hora de comunicar qué hacen, pero asuma que quién está leyendo su código no es tan listo como usted (o, si lo es, que ha pasado una mala noche). De nuevo, para ver cómo se documenta bien una clase, puede inspirarse en cómo se ha hecho esto en `Node`.

3. Análisis de un algoritmo recursivo

Antes de que programe usted mismo un algoritmo recursivo, va a analizar uno que ya está implementado: el que se utiliza para dibujar los `Nodes` en pantalla. Abra la clase `SkeletonPanel`, que se encuentra en el paquete `es.upm.aled.lab2.gui`, y dedique un tiempo a leerla e intentar entender lo que hace. Este proceso le hará ser consciente de lo importantes que son los comentarios, si es que no estaba convencido aún de ello.

Responda a las siguientes preguntas:

- ¿Qué tareas realiza el constructor?
- ¿Qué hace el método `paintComponent()`?
- ¿A qué dos métodos, que representan cada uno un algoritmo recursivo distinto, llama el método `paintComponent()`? ¿Cuál de ellos ya está implementado?
- ¿Alguno de los métodos de `SkeletonPanel` se llama a sí mismo?

3.1. Documentación del código

Llegados a este punto, ya se habrá dado cuenta de que el método `drawSkeleton()` contiene la implementación de un algoritmo recursivo: el que pinta en pantalla cada uno de los `Nodes` de los que se compone el dibujo de un exoesqueleto. Puede haberse percatado de esto siguiendo uno de los tres acercamientos siguientes:

1. Ha razonado que todos los `Nodes` están conectados con, al menos, otro. Eso significa que, para poder dibujar la línea que llega hasta un `Node` nuevo será necesario haber dibujado primero el otro `Node` (exceptuando el primer `Node`, cuya línea va al origen).
2. Ha intuido que los `Nodes` están relacionados siguiendo una estructura de árbol, por lo que si se quiere obtener la información de un `Node` concreto es preciso llamar al método `getChildren()` de su padre. Pero claro, para obtener la información de este padre, primero será necesario haber llamado al método `getChildren()` del “abuelo”, y así sucesivamente, repitiendo este proceso tantas veces como sea necesario.
3. Ha leído el código y se ha dado cuenta de que `drawSkeleton()` llama a `drawSkeleton()` (posiblemente más de una vez, pues lo hace dentro de un `for`), pasándole unos parámetros distintos a lo que recibió. Esto significa que cada `drawSkeleton()` solo retornará cuando lo hayan hecho todos los `drawSkeleton()` a los que este ha llamado.

DIFERENTES FORMAS DE PENSAR

No importa cuál de las líneas de razonamiento haya seguido para llegar a la conclusión de que `drawSkeleton()` implementa un algoritmo recursivo. Cada persona tiene una manera de razonar que le resulta más intuitiva que el resto, por lo que suele recurrir a ella para llegar a las conclusiones correctas lo más fácilmente posible. Quizá usted tenga un pensamiento más “algorítmico” y haya razonado como se indica en el acercamiento 3, o quizá prefiera visualizar las cosas y por eso haya seguido el 2. En cualquier caso, no importa; bien está lo que bien acaba.

Eso sí, si puede, intente entender el porqué de los razonamientos que no ha seguido, para ir ejercitando su mente y acostumbrándose a pensar en diferentes formas de idear soluciones a problemas reales.

Responda a las siguientes preguntas:

- ¿Qué líneas de código representan el caso base?
- ¿Qué líneas de código representan el paso recursivo?
- ¿Qué líneas de código representan el código general o común?
- ¿Qué se dibuja primero, el final (extremo final del último segmento) o el principio (punto inicial del segmento raíz) de la cadena cinemática? Le recomiendo recurrir al depurador para confirmar si su respuesta es correcta o no.

Por último, ahora que sabe para qué sirve cada una de las líneas de código de `drawSkeleton()`, añada tantos comentarios como sea necesario (de nuevo, cuantos más mejor) al interior de este método, para que quede claro cómo funciona.

4. Implementación de un algoritmo recursivo

Ahora que ya ha analizado un algoritmo recursivo, ha llegado el momento de que implemente el suyo propio. Cuando analizó el método `drawSkeleton()` de la clase `SkeletonPanel`, observó que llamaba al método `computePositions()` de la clase `ForwardKinematics`. Abra ahora esta clase, porque tendrá que implementar los dos métodos que contiene.

Antes de continuar leyendo, repase el procedimiento para la resolución del problema de la cinemática directa en 2 dimensiones que aparecía en la sección 1.4. Lo que hará a continuación será precisamente implementar dicho algoritmo.

4.1. Implementar el método fachada

En muchas implementaciones de algoritmos recursivos, el método al que se llama recursivamente necesita una serie de argumentos “adicionales”, que van cambiando en cada llamada a dicho método pero que el usuario que inicia el algoritmo no tiene que proporcionar.

En problemas en los que se da esta circunstancia, es habitual separar la implementación en:

- Un **método recursivo** privado, que es la implementación del algoritmo recursivo propiamente dicho.
- Un **método fachada** público, al que llama el que invoca el algoritmo recursivo. Este método fachada hace la primera llamada al método recursivo.

En el ejemplo que nos ocupa, el método fachada recibe como argumentos:

- **root**: El `Segment` raíz del árbol de segmentos.
- **originX** y **originY**: Las coordenadas del punto inicial del segmento raíz.

Mientras que el **método privado recibe, además de estos argumentos, un `accumulatedAngle`**, que se va incrementando según se va avanzando por el árbol de segmentos. De nuevo, revise las **ecuaciones de la sección 1.4** para entender por qué el ángulo se va acumulando.

Implemente el método fachada, que deberá llamar al método privado pasándole los mismos argumentos que ha recibido y un **`accumulatedAngle` de 0**, ya que al principio del algoritmo no hay ningún ángulo previo que sumar al actual.

4.2. Implementar el método privado

Ahora implemente el auténtico algoritmo recursivo dentro del método privado. Este apartado le llevará la mayor parte del trabajo de esta práctica, así que dedíquelo todo el tiempo que sea necesario, no se frustre, y recurra tanto al papel como al depurador cuantas veces sea necesario.

Para resolver con éxito esta parte de la práctica, tenga en cuenta lo siguiente:

- El método que está implementando devuelve un `Node`, no un `Segment` (`Segment` es lo que recibe como argumento).
- El método que está implementando devuelve un `Node`. Esto significa que no es necesario devolver una `List<Node>` o ninguna otra colección similar, porque cada `Node` ya contiene una `List` con sus hijos.

- Hablando de lo cual... No olvide añadir cada **Node** a la lista de hijos de su **Node** padre después de haber calculado su posición. El único **Node** que no tendrá hijos será el último de cada rama del árbol.
- Uno de los aspectos más importantes a la hora de implementar un algoritmo usando recursividad es el caso base. Para este problema, el caso base deberá ocurrir cuando el **Segment** que se pasa al método recursivo no tenga hijos. Si se da esta circunstancia, deberá construirse el **Node** pertinente, pero este no tendrá tampoco hijos.
- En el paso recursivo, ponga cuidado en que los argumentos que se pasa a la nueva llamada al método son los correctos. Un error muy común es pasar otra vez los argumentos que ya se habían recibido de la anterior llamada.
- Piense en qué aspectos del algoritmo pueden considerarse código general. Serán siempre aquellos cálculos que sea necesario realizar en todas las llamadas, tanto si estamos hablando del caso base como de los pasos recursivos intermedios.

4.3. Ejecutar el método *main*

Para comprobar que el código que ha escrito funciona, se ha preparado una clase **SkeletonVisualizer**, cuyo método **main** lleva a cabo las siguientes tareas:

1. Construye una cadena cinemática que representa un exoesqueleto de cuerpo entero con 10 segmentos: torso, cuello, brazo izquierdo, antebrazo izquierdo, brazo derecho, antebrazo derecho, muslo izquierdo, pantorrilla izquierda, muslo derecho y pantorrilla derecha.
2. Abre una ventana y representa gráficamente este exoesqueleto.
3. Si se le pasa un argumento al método **main**, abrirá el archivo cuya ubicación coincida con este argumento. Dicho archivo deberá estar formateado en CSV (como en la práctica 1) y contener una secuencia de movimientos *frame a frame* para 8 de las extremidades de este exoesqueleto. Junto con la práctica se le proporciona un archivo con una secuencia de movimientos de ejemplo, llamado *walking_cycle.csv*, que se encuentra en la carpeta *animations*.
4. Si ha conseguido cargar el contenido de un archivo, el programa anima el exoesqueleto en un bucle de rebote (primero en un sentido, luego en otro, luego de vuelta al sentido original, y así *ad infinitum*) a partir de los datos de movimiento recibidos.

Utilice este **main** para poner a prueba el código que ha escrito. Si ha implementado correctamente el algoritmo recursivo, debería ver una ventana como la de la Figura 4. Si no, es que ha hecho algo mal y habrá llegado el momento de depurar.

Recuerde que, por defecto, al ejecutar el método **main** de una clase, Eclipse no le pasa ningún argumento. Por tanto, si hace una ejecución directa el programa simplemente representará el exoesqueleto de forma estática, sin animarlo. Por tanto, si quiere que su exoesqueleto se mueva (se lo recomiendo, la verdad es que resulta gracioso), tendrá que crear una configuración de lanzamiento, como ya hizo en la práctica 1.

Responda a las siguientes preguntas:

- ¿Cómo debe modificar el código del **main** para que los segmentos sean más largos o más cortos?
- ¿Cómo debe modificar el código del **main** para añadir tres dedos a cada mano?

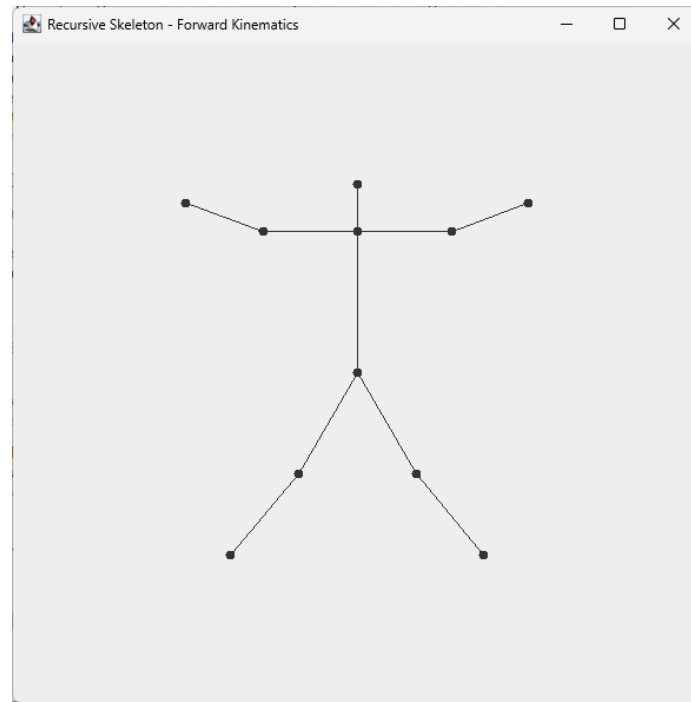


Figura 4: Dibujo del exoesqueleto de ejemplo

5. Medida de tiempos

Más adelante, en esta asignatura, verá lo importante que es analizar el rendimiento de un algoritmo. Para ir abriéndole el apetito, instrumentalizará (de forma bastante inocente) su código, de tal manera que pueda observar el tiempo que lleva la ejecución de cada invocación del método `computePositions()` (la versión privada, no la fachada) que usted mismo ha programado.

La forma de hacer esto será anotar el instante de tiempo en el que comienza el método y el instante de tiempo en el que termina y restar ambos valores. El resultado será el tiempo que ha tardado el método en ejecutarse. En Java, puede obtener el instante de tiempo en nanosegundos en el que ocurre algo mediante el método estático `nanoTime()` de la clase `System`.

Por tanto, escriba el siguiente código al principio del cuerpo del método `computePositions()`:

```
long startTime = System.nanoTime();
```

Y el siguiente código al final, justo antes del `return` (¡tenga en cuenta que puede haber varios `return`!):

```
long runningTime = System.nanoTime() - startTime;
System.out.println("Tiempo de computePositions para un segmento con "
    + link.getChildren().size() + " hijos: "
    + runningTime + " nanosegundos");
```

Después, ejecute el código y fíjese en lo que aparece en el terminal.

Responda a las siguientes preguntas:

- ¿Cómo varía el tiempo de ejecución en función del número de hijos del `Segment`?
- ¿Tiene alguna teoría de por qué ocurre esto?

Si siente curiosidad, instrumentalice otras partes de su código. Enhorabuena, ha terminado la práctica.

6. Extras

Si siente curiosidad, pruebe a llevar a cabo las tareas siguientes:

- Averigüe lo que significa el `@Override` que aparece encima de `paintComponent()` en la clase `SkeletonPanel`.
- Averigüe lo que significa el operador `->` en Java. Aparece en el `main`, en la línea en la que se crea el `Timer` que controla la animación.
- Cree su propio archivo de secuencia de movimientos, que ejecute algún movimiento curioso como mover un solo brazo, hacer un saludo militar o dar saltitos.
- Modifique el código de la práctica de tal forma que sea imposible añadir un `Node` o un `Segment` al árbol correspondiente más de una vez. ¡Ojo! No es suficiente con comprobar que no aparece en la misma lista de hijos dos veces (como ya ha hecho en la sección 2.1), sino también que no se encuentra **en ninguna otra** lista de hijos del árbol. Esto es necesario para impedir ciclos (lo que convertiría el árbol en un grafo), que acaban produciendo bucles infinitos.
- Investigue lo que es la cinemática **inversa**.

OTRAS APLICACIONES DEL ANÁLISIS BIOMECÁNICO

El análisis biomecánico es fundamental para el diseño de exoesqueletos y prótesis, pero esta no es su única aplicación. Otras aplicaciones biomédicas que hacen uso de la información recogida mediante el modelado del movimiento humano son (sin ser exhaustivos):

- Análisis de la marcha.
- Diseño de tratamientos de fisioterapia.
- Entrenamiento de deportistas de élite.
- Diagnóstico temprano de problemas fisiológicos.
- Planificación de intervenciones quirúrgicas.
- Análisis postural.
- Robótica quirúrgica.